

ESWIN

ESSDK 安全算法用户手册

Rev 1.0

2026-05-21

声明

知识产权声明

除非另有说明，ESWIN 保留对本产品的所有知识产权。未经 ESWIN 授权，不得对本产品进行反向工程、反编译，不得修改、改编、制作、分发本产品的衍生作品。

ESWIN 保留更新本文档或改进其中所提及的产品或服务的权利。除非另有说明，本文档中的所有陈述、信息及建议均按“现状”提供，ESWIN 不对此做出任何形式的担保、保证或承诺，无论明示或默示。

“ESWIN”商标和其他 ESWIN 标识，为北京奕斯伟计算技术股份有限公司及（或）其母公司所有的商标，未经授权，不得擅自使用。

责任限制

除本文档或交易文书另有约定外，ESWIN 不提供任何明示、暗示或法律上的保证，包括但不限于适销性、适用性、所有权或非侵权的任何暗示或明示保证，或因任何建议、规格、样品、交易、惯例或贸易惯例而产生的任何保证。

ESWIN 不对任何意外性、惩罚性、间接性的损害承担责任，包括但不限于利润损失、业务中断或采购任何替代商品或服务而产生的成本。

修订记录

文档版本	日期	说明
V1.0	2026-05-21	完善附录与概述章节，规范全文格式
V0.5	2024-11-29	规范产品品牌名称，修订文档标题
V0.1	2023-12-04	初始版本

目录

声明	1
知识产权声明	1
责任限制	1
修订记录	2
1. 概述	5
1.1. 密码算法使用流程	6
1.1.1. 对称加解密	6
1.1.2. HASH 计算	6
1.1.3. HMAC 计算	6
1.1.4. RSA 加解密	7
1.1.5. RSA 签名与验签	7
1.1.6. NsignVerify 签名验证	8
1.1.7. ECDH 密钥协商	8
1.2. 辅助功能使用流程	9
1.2.1. 随机数生成	9
1.2.2. RAM 公钥更新	9
1.2.3. Image 解密	10
1.2.4. 扩展服务程序下载	10
2. API 参考	11
2.1. ES_CIPHER_Init	11
2.2. ES_CIPHER_DeInit	11
2.3. ES_CIPHER_CreateHandle	11
2.4. ES_CIPHER_DestroyHandle	12
2.5. ES_CIPHER_Encrypt	12
2.6. ES_CIPHER_Decrypt	13
2.7. ES_CIPHER_GetHandleConfig	14
2.8. ES_CIPHER_HashInit	14
2.9. ES_CIPHER_HashFinish	15
2.10. ES_CIPHER_GetRandomNumber	15
2.11. ES_CIPHER_RsaPublicEncrypt	16
2.12. ES_CIPHER_RsaPrivateDecrypt	16
2.13. ES_CIPHER_RsaPrivateEncrypt	17
2.14. ES_CIPHER_RsaPublicDecrypt	18
2.15. ES_CIPHER_NSignVerify	18
2.16. ES_CIPHER_ImageDec	19
2.17. ES_CIPHER_EcdhKeyGen	20
2.18. ES_CIPHER_EcdhKeyAgree	20
2.19. ES_CIPHER_EcdhKeyDerivation	21
2.20. ES_CIPHER_PubKeyDownload	21
2.21. ES_CIPHER_LoadableRegister	22
2.22. ES_CIPHER_LoadableUnRegister	23
2.23. ES_CIPHER_LoadableIoctl	23
2.24. ES_CIPHER_RsaSign	24
2.25. ES_CIPHER_RsaVerify	25
2.26. ES_CIPHER_NsignFileParse	25
3. 数据结构	27
3.1. ES_HANDLE	27
3.2. ES_CIPHER_ALG_MODE_E	27
3.3. ES_CIPHER_ENC_KEY_LENGTH_E	28
3.4. ES_CIPHER_SYMC_KEY_SRC_E	28
3.5. ES_CIPHER_CTRL_S	29

3.6. ES_CIPHER_HASH_CTRL_S 29

3.7. ES_CIPHER_RSA_ENC_SCHEME_E 29

3.8. ES_CIPHER_RSA_KEY_SRC_E 30

3.9. ES_CIPHER_RSA_PUB_KEY_S 30

3.10. ES_CIPHER_RSA_PUB_CTRL_S 30

3.11. ES_CIPHER_RSA_PRI_KEY_S 31

3.12. ES_CIPHER_RSA_PRI_CTRL_S 31

3.13. ES_CIPHER_NSIGN_VERIFY_S 31

3.14. ES_CIPHER_ECC_CURVE_TYPE_E 31

3.15. ES_CIPHER_ECDH_KEY_S 32

3.16. ES_CIPHER_ECDH_PUB_KEY_S 32

3.17. ES_CIPHER_ECDH_PRI_KEY_S 32

4. 错误码 **33**

1. 概述

CIPHER 是奕斯伟 EIC7x 系列芯片提供的安全算法模块，其提供了包括 AES、DES/3DES 和 SM4 等对称加解密算法，RSA、ECDSA 不对称加解密算法，随机数生成，以及支持 HASH、HMAC、MAC 等摘要算法，主要用于对音视频码流进行加解密保护，用户合法性认证等场景。不同算法支持的工作模式如下表所示：

算法	支持的工作模式
AES	ECB/CBC/CTR/CFB/OFB/CS1/CS2/CS3；注：通过加载 service code 的方式，可以支持 CCM/GCM 模式
DES/3DES	CBC/ECB
SM4	ECB/CBC/CFB/OFB/CTR/CS1/CS2/CS3；注：通过加载 service code 的方式，可以支持 CCM/GCM 模式
RSA	1024/2048/4096
ECDSA	NIST_P256/ NIST_P384/ NIST_P521/ BRAINPOOL_P256R1/ BRAINPOOL_P384R1/ BRAINPOOL_P512R1
HASH	SHA1/SHA224/SHA256/SHA384/SHA512/SHA512_224/SHA512_256/MD5/SM3
HMAC	HMAC_SHA1/HMAC_SHA224/HMAC_SHA256/HMAC_SHA384/HMAC_SHA512/HMAC_SHA512_224/HMAC_SHA512_256/HMAC_MD5/HMAC_SM3
随机数	TRNG

CIPHER 模块提供了 NSIGN 验签功能。需要配合使用奕斯伟提供的 NSIGN 签名打包工具生成签名。

CIPHER 模块还提供了 NSIGN 包解密功能。该功能先验证签名，如果数据是密文，会同时解密生成明文。需要配合使用奕斯伟提供的 NSIGN 签名打包工具生成签名及对数据加密。

CIPHER 模块还支持加载和运行一段经过验签的 firmware。Firmware 需要使用 NSIGN 签名打包工具生成签名。

上述算法及功能都是以 service 服务的方式，向用户提供服务 and 接口。CIPHER 模块也支持加载和运行用户的 service code，从而提供用户自定义的 service 功能。该 service code 同样需要使用 NSIGN 签名打包工具生成签名。如图所示：

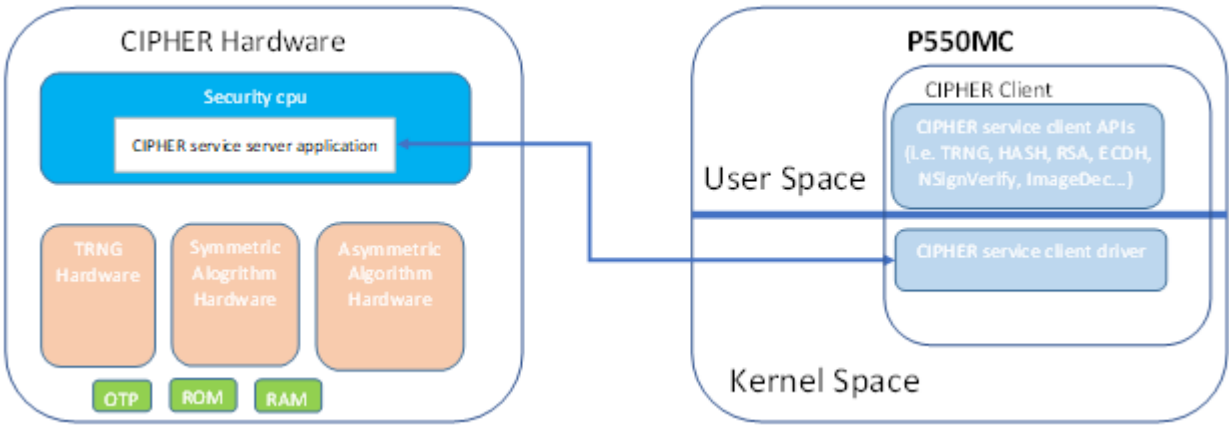


图 1: CIPHER 框架

Security CPU 启动时，会读取 OTP Security Enable 标记位。如果标记位是 security mode，SCPU 会执行 security boot 流程，即下一级 boot image(一般是 opensbi+uboot) 做验签，下一级 boot image 必须是密文，验签和解密通过后才会执行 boot image。同时，如果是 security mode，本文所描述的所有 service 服务都支持。如果标记位是 un-security mode，SCPU 不会对下一级 boot image 做验签，下一级 boot image 必须是明文。同时，如果是 un-security mode，以下四个 service 服务不支持：NsignVerify 签名验证、Image 解密、更新 RAM 公钥、下载扩展服务程序。

1.1. 密码算法使用流程

1.1.1. 对称加解密

工作流程

对数据进行对称的 DES/3DES/AES/SM4 加解密的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 配置 CIPHER 控制信息，包含密钥、初始向量、加密算法、工作模式等信息，并获取该控制信息的 CIPHER 句柄。调用接口 ES_CIPHER_CreateHandle 完成。
3. 对数据进行加解密。
 - 加密- ES_CIPHER_Encrypt 当拆分成多个包加密时，多次调用该接口。
 - 解密- ES_CIPHER_Decrypt 当拆分成多个包解密时，多次调用该接口。
4. 销毁 CIPHER 句柄。调用接口 ES_CIPHER_DestroyHandle 完成。
5. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. 对称加解密操作中每个数据包的长度不能大于 1GB。如果数据长度大于 1G，请拆分成多个数据包进行处理。
2. 传入的地址参数为用户空间虚拟地址。
3. 加密、解密的源地址、目的地址可以相同，即可以在原地（密文、明文使用同一块 buffer）进行数据的加密和解密。
4. 在进行加密、解密运算前必须先获取 CIPHER 句柄，当长时间不使用时可以释放，建议加密、解密各获取一个句柄，每个句柄只进行加密或者只进行解密操作。
5. 使用非 ECB 模式进行 CIPHER 的加解密，获取控制信息的 CIPHER 句柄时，需要传入第一个包的初始化向量 IV (Initial Vector)。拆分成多个数据包加解密时，对每个包调用 ES_CIPHER_Encrypt/ES_CIPHER_Decrypt，上一个包的向量运算结果会自动作为下一个包的 IV。
6. 原生不支持 CCM 和 GCM 两种模式。可通过扩展服务程序，支持这两种模式。

示例

具体示例请参见发布包 sample: sample_aes_sm4.c

1.1.2. HASH 计算

工作流程

对数据进行 HASH 计算的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 选择 HASH 算法，获取 HASH 句柄。调用接口 ES_CIPHER_HashInit 完成。
3. 输入数据，完成 HASH 计算。调用接口 ES_CIPHER_HashFinish 完成。
4. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. HASH 计算输入数据最大 size 为 1GB，不支持分成多个数据块依次计算 HASH 值。
2. 传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_hash.c

1.1.3. HMAC 计算

场景说明

计算数据的 HMAC 值。HMAC 是密钥相关的哈希运算消息认证码 (Hash-based Message Authentication Code) 的缩写。HMAC 中的 H 指 hash 散列算法，HMAC 可以使用多种散列算法其计算结果长度与对应散列算法一致。CIPHER 硬件支持的散列算法如下：

MD5、SHA1、SHA224、SHA256、SHA384、SHA512、SHA512_224、SHA512_256、SM3、XCBC、CMAC

工作流程

对数据进行对称的 HMAC 计算的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 选择 HASH 算法，输入 HMAC key 和 keyLen，获取 HASH 句柄。调用接口 ES_CIPHER_HashInit 完成。
3. 输入数据，完成 HMAC 计算。调用接口 ES_CIPHER_HashFinish 完成。
4. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. HMAC 计算输入数据最大 size 为 1GB，不支持分成多个数据块依次计算 HMAC 值。
2. 传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_hash.c

1.1.4. RSA 加解密

场景说明

对数据进行 RSA 非对称加解密。当使用公钥加密的数据，必须使用私钥进行解密，反之，使用私钥加密的数据，必须使用公钥进行解密。

工作流程

对数据进行 RSA 非对称加解密的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 根据使用的密钥不同，分为 4 个接口，用户可以调用以下任一接口进行加解密。
 - 公钥加密 ES_CIPHER_RsaPublicEncrypt
 - 私钥解密 ES_CIPHER_RsaPrivateDecrypt
 - 私钥加密 ES_CIPHER_RsaPrivateEncrypt
 - 公钥解密 ES_CIPHER_RsaPublicDecrypt
3. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. RSA 的密钥位宽可选 1024、2048、4096。加密时，采用 RSAES-PKCS1-v1_5 方式对明文做 padding。所以明文的长度必须小于 [密钥的长度 - 11]。
2. 传入的地址参数为用户空间虚拟地址。
3. CIPHER 硬件内部的 ROM 上，固化了 7 个 RSA2048 公钥，不能被更改。
4. CIPHER 硬件内部的 OTP 上，可存储 1 个 RSA2048 公钥，只能烧写一次。
5. CIPHER 硬件内部的 ram 上，可存储 1 个 RSA2048 公钥，可由用户更新，更新次数不限。
6. CIPHER 硬件，可使用来自用户传入的公钥或者私钥（支持 1024、2048、4096 位宽），进行 RSA 加解密。

示例

具体示例请参见发布包 sample: sample_rsa.c

1.1.5. RSA 签名与验签

场景说明

RSA 签名时，首先计算输入数据的 SHA256 值，再使用私钥对 SHA256 值进行加密得到签名。RSA 验签时，首先计算输入数据的 SHA256 值 A，再使用公钥对签名解密得到解密 SHA256 值 B，比较 A 和 B 的值，若相同则验签成功，否则验签失败。

工作流程

RSA 签名及验签过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 对数据进行签名或签名验证，调用以下接口完成。
 - 私钥签名- ES_CIPHER_RsaSign
 - 公钥验证- ES_CIPHER_RsaVerify
3. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

RSA 公钥可以来自外部或者内部。RSA 私钥只能来自外部输入。当密钥来自外部时，位宽可选 1024、2048、及 4096。当 RSA 公钥来自内部 OTP/ROM/RAM 时，位宽只能选 2048。

示例

具体示例请参见发布包 sample: sample_rsa_sign.c

1.1.6. NsignVerify 签名验证

场景说明

NsignVerify 签名验证功能需要配合 PC 上的 ESWIN NSIGN 工具一同使用。首先，通过 ESWIN NSIGN 工具对 image 打包，生成包含 sign、image 的 bin 文件。再由 NsignVerify 验证 bin 文件的合法性，输出明文签名。

工作流程

对 NSIGN 工具打包后的 bin 文件，进行签名验签的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 从 bin 文件中获取签名和 image 的地址。调用 ES_CIPHER_NsignFileParse 接口完成。
3. 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 image 地址，进行签名验证。调用 ES_CIPHER_NsignVerify 完成。
4. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. 签名验证使用的公钥，只能来自 CIPHER 硬件内部的 OTP、ROM 或者 RAM 上预先存储的公钥。如果 RAM 上预先未存储公钥，则不能选择 RAM key 作为参数，否则会导致 CIPHER 硬件工作异常。
2. CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。
3. 传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_nsign_verify.c

1.1.7. ECDH 密钥协商

场景说明

ECDH（椭圆曲线迪菲-赫尔曼密钥交换）是一种密钥协商协议，其核心是生成双方通信的对称密钥，对称密钥生成过程中需要双向进行公钥交换，对称密钥生成后，双方基于对称密钥进行对称加密通信。

工作流程

ECDH 方式生成对称密钥的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 选择椭圆曲线类别，生成非对称密钥。调用 ES_CIPHER_EcdhKeyGen 接口完成。
3. 根据远端的公钥，结合本地的公私钥，协商出共享的公钥。调用 ES_CIPHER_EcdhKeyAgree 接口完成。
4. 根据远端的随机数，结合步骤 3 协商出的共享公钥，计算对称密钥。调用 ES_CIPHER_EcdhKeyDerivation 接口完成。
5. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. 椭圆曲线的类型支持 NIST_P256、NIST_P384、NIST_P521、BRAINPOOL_P256R1、BRAINPOOL_P384R1、BRAINPOOL_P512R1。
2. 传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_ecdh.c

1.2. 辅助功能使用流程

1.2.1. 随机数生成

场景说明

获取硬件产生的真随机数。

工作流程

生成随机数的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 传入随机数的长度（以 byte 为单位）以及随机数输出的地址，获取随机数。调用接口 ES_CIPHER_GetRandom-Number 完成。
3. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

传入的地址参数为数据的用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_trng.c

1.2.2. RAM 公钥更新

场景说明

CIPHER 硬件内部 RAM 放置了一组 RSA 和一组 ECDSA 临时的公钥。其中 RSA 公钥可以用于数据的加解密或签名验签。ECDSA 公钥可以用于签名验签。这两组公钥可由用户更新，更新过程中，这两组公钥需要通过 OTP 或 ROM 上的公钥进行签名验证。首先，通过 PC 上的 NSIGN 工具，对需要更新的 RAM 公钥进行签名打包生成 bin 文件。通过该 API 接口验签后更新 CIPHER 内部的 RAM 公钥。

工作流程

更新 CIPHER 内部 RAM 公钥的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 从 bin 文件中获取签名和 payload 的地址。调用 ES_CIPHER_NsignFileParse 接口完成。
3. 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 payload 地址，更新 RAM 上的公钥。调用 ES_CIPHER_PubKeyDownload 完成。
4. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. 验签使用的公钥，只能来自 CIPHER 硬件内部的 OTP 或者 ROM 上预先存储的公钥。
2. CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。
3. 传入的地址参数为用户空间虚拟地址。
4. CIPHER 硬件会把 payload 解密后的内容，覆盖到输入的 payload 地址。因此，请用户保存好原始 payload 的备份。

1.2.3. Image 解密

场景说明

Image 解密需要配合 PC 上的 ESWIN NSIGN 工具一同使用。Image 解密的过程是在 NsignVerify 的基础上，继续对 bin 文件里面的 image 解密，得到明文数据。

工作流程

对 NSIGN 工具打包后的 bin 文件，进行 image 解密的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 从 bin 文件中获取签名和 image 的地址。调用 ES_CIPHER_NsignFileParse 接口完成。
3. 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 image 地址，进行 image 解密。调用 ES_CIPHER_ImageDec 完成。
4. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. Image 解密使用的公钥，只能来自 CIPHER 硬件内部的 OTP、ROM 或者 RAM 上预先存储的公钥。如果 RAM 上预先未存储公钥，则不能选择 RAM key 作为参数，否则会导致 CIPHER 硬件工作异常。
2. CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。
3. 传入的地址参数为用户空间虚拟地址。

示例

具体示例请参见发布包 sample: sample_image_dec.c

1.2.4. 扩展服务程序下载

场景说明

本文所涉及的 API 都是与 CIPHER 硬件中运行的 service 服务程序通信，由 service 服务程序实际完成 API 请求的功能。用户也可以下载一段自己的 service 到 CIPHER 硬件中运行，实现扩展服务功能。例如，可通过扩展服务程序，支持 CCM 和 GCM 模式的对称加解密功能。该功能需要配合 PC 上的 ESWIN NSIGN 工具一同使用。首先，通过 ESWIN NSIGN 工具对 image 打包，生成包含 sign、image 的 bin 文件。经过验签后，image 作为 service 服务程序下载到 CIPHER 硬件运行。在下载的服务程序中，至少需要实现 4 个函数接口：

- Initialize function-用于服务注册
- Destroy function-用于服务注销
- ioctl function-处理用户的输入
- Irqfunction-中断服务函数 (可选)

工作流程

下载、调用、注销扩展服务的过程如下：

1. CIPHER 设备初始化。调用接口 ES_CIPHER_Init 完成。
2. 从 bin 文件中获取签名和 image 的地址。调用 ES_CIPHER_NsignFileParse 接口完成。
3. 选择非对称加解密算法，以及公钥在 CIPHER 硬件中存储的位置，输入签名地址和 image 地址，验签后下载 image 到 CIPHER 硬件并运行。调用 ES_CIPHER_LoadableRegister 完成。
4. 调用 ES_CIPHER_LoadableIoctl，处理用户的输入。
5. 调用 ES_CIPHER_LoadableUnRegister，注销服务。
6. 关闭 CIPHER 设备。调用接口 ES_CIPHER_DeInit 完成。

— 结束

注意事项

1. 验签使用的公钥，只能来自 CIPHER 硬件内部的 OTP、ROM 或者 RAM 上预先存储的公钥。如果 RAM 上预先未存储公钥，则不能选择 RAM key 作为参数，否则会导致 CIPHER 硬件工作异常。
2. CIPHER 硬件内部的公钥，只支持 RSA2048 和 ECDSA 两种非对称算法。
3. 传入的地址参数为用户空间虚拟地址。

2. API 参考

Cipher API 的库文件 libes_cipher.so, libes_cipher.a, 头文件: es_cipher.h

2.1. ES_CIPHER_Init

【函数体】

```
ES_S32 ES_CIPHER_Init(ES_VOID)
```

【描述】

初始化 CIPHER 设备。

【参数】

无

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

2.2. ES_CIPHER_DeInit

【函数体】

```
ES_S32 ES_CIPHER_DeInit(ES_VOID)
```

【描述】

去初始化 CIPHER 设备。

【参数】

无

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

2.3. ES_CIPHER_CreateHandle

【函数体】

```
ES_S32 ES_CIPHER_CreateHandle(  
    ES_HANDLE *phCipher,  
    ES_CIPHER_CTRL_S *pstCipherAttr)
```

【描述】

获取用于对称加密解密的 CIPHER 句柄。

【参数】

参数名	描述	输入/输出
phCipher	CIPHER 句柄	输出
pstCipherAttr	对称算法属性	输入

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

2.4. ES_CIPHER_DestroyHandle

【函数体】

```
ES_S32 ES_CIPHER_DestroyHandle(  
    ES_HANDLE hCipher)
```

【描述】

销毁已存在的 CIPHER 句柄。

【参数】

参数名	描述	输入/输出
hCipher	CIPHER 句柄	输入

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

2.5. ES_CIPHER_Encrypt

【函数体】

```
ES_S32 ES_CIPHER_Encrypt(  
    ES_HANDLE hCipher,  
    ES_U8 *pu8SrcData,  
    ES_U8 *pu8DestData,  
    ES_U32 u32ByteLength)
```

【描述】

执行对称加密。

【参数】

参数名	描述	输入/输出
hCipher	CIPHER 句柄	输入
pu8SrcData	源数据缓冲区	输入
pu8DestData	目标数据缓冲区	输出
u32ByteLength	源数据长度	输入

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

提示

源数据长度必须按 16 字节对齐。

2.6. ES_CIPHER_Decrypt**【函数体】**

```
ES_S32 ES_CIPHER_Decrypt(
    ES_HANDLE hCipher,
    ES_U8 *pu8SrcData,
    ES_U8 *pu8DestData,
    ES_U32 u32ByteLength)
```

【描述】

执行对称解密。

【参数】

参数名	描述	输入/输出
hCipher	CIPHER 句柄	输入
pu8SrcData	源数据缓冲区	输入
pu8DestData	目标数据缓冲区	输出
u32ByteLength	源数据长度	输入

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

提示

源数据长度必须按 16 字节对齐。

2.7. ES_CIPHER_GetHandleConfig

【函数体】

```
ES_S32 ES_CIPHER_GetHandleConfig(  
    ES_HANDLE hCipher,  
    ES_CIPHER_CTRL_S *pstCtrl)
```

【描述】

获取 CIPHER 控制信息。

【参数】

参数名	描述	输入/输出
hCipher	CIPHER 句柄	输入
pstCtrl	CIPHER 控制信息	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.8. ES_CIPHER_HashInit

【函数体】

```
ES_S32 ES_CIPHER_HashInit(  
    ES_CIPHER_HASH_CTRL_S *pstHashCtrl,  
    ES_HANDLE *pHashHandle)
```

【描述】

选择 HASH 算法并返回 HASH 句柄。

【参数】

参数名	描述	输入/输出
pstHashCtrl	HASH 计算结构输入	输入
pHashHandle	输出 HASH 句柄	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.9. ES_CIPHER_HashFinish

【函数体】

```
ES_S32 ES_CIPHER_HashFinish(  
    ES_HANDLE hHashHandle,  
    ES_U8 *pu8InputData,  
    ES_U32 u32InputDataLen,  
    ES_U8 *pu8OutputHash,  
    ES_U32 *pu32Hlen)
```

【描述】

计算 HASH 值。

【参数】

参数名	描述	输入/输出
hHashHandle	HASH 句柄	输入
pu8InputData	输入数据缓冲区	输入
u32InputDataLen	输入数据长度	输入
pu8OutputHash	输出 HASH 结果	输出
pu32Hlen	输出 HASH 结果长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

- 输入数据的最大长度为 1GB
- 不支持分片，即不支持 HashUpdate()

2.10. ES_CIPHER_GetRandomNumber

【函数体】

```
ES_S32 ES_CIPHER_GetRandomNumber(  
    ES_U8 *pu8RandomNumber,  
    ES_U32 u32ByteLength)
```

【描述】

从硬件获取真随机数。

【参数】

参数名	描述	输入/输出
pu8RandomNumber	随机数据输出	输出
u32ByteLength	随机数据长度	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.11. ES_CIPHER_RsaPublicEncrypt

【函数体】

```
ES_S32 ES_CIPHER_RsaPublicEncrypt(  
    ES_CIPHER_RSA_PUB_CTRL_S *pstRsaEnc,  
    ES_U8 *pu8Input,  
    ES_U32 u32InLen,  
    ES_U8 *pu8Output,  
    ES_U32 *pu32OutLen)
```

【描述】

使用 RSA 公钥对明文进行加密。

【参数】

参数名	描述	输入/输出
pstRsaEnc	公钥加密结构	输入
pu8Input	待加密的输入数据	输入
u32InLen	待加密输入数据长度	输入
pu8Output	输出加密数据	输出
pu32OutLen	输出加密数据长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.12. ES_CIPHER_RsaPrivateDecrypt

【函数体】

```
ES_S32 ES_CIPHER_RsaPrivateDecrypt(
    ES_CIPHER_RSA_PRI_CTRL_S *pstRsaDec,
    ES_U8 *pu8Input,
    ES_U32 u32InLen,
    ES_U8 *pu8Output,
    ES_U32 *pu32OutLen)
```

【描述】

使用 RSA 私钥对密文进行解密。

【参数】

参数名	描述	输入/输出
pstRsaDec	私钥解密结构	输入
pu8Input	待解密的输入数据	输入
u32InLen	待解密输入数据长度	输入
pu8Output	输出解密数据	输出
pu32OutLen	输出解密数据长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.13. ES_CIPHER_RsaPrivateEncrypt

【函数体】

```
ES_S32 ES_CIPHER_RsaPrivateEncrypt(
    ES_CIPHER_RSA_PRI_CTRL_S *pstRsaEnc,
    ES_U8 *pu8Input,
    ES_U32 u32InLen,
    ES_U8 *pu8Output,
    ES_U32 *pu32OutLen)
```

【描述】

使用 RSA 私钥对明文进行加密。

【参数】

参数名	描述	输入/输出
pstRsaEnc	私钥加密结构	输入
pu8Input	待加密的输入数据	输入
u32InLen	待加密输入数据长度	输入
pu8Output	输出加密数据	输出
pu32OutLen	输出加密数据长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.14. ES_CIPHER_RsaPublicDecrypt

【函数体】

```
ES_S32 ES_CIPHER_RsaPublicDecrypt(  
    ES_CIPHER_RSA_PUB_CTRL_S *pstRsaDec,  
    ES_U8 *pu8Input,  
    ES_U32 u32InLen,  
    ES_U8 *pu8Output,  
    ES_U32 *pu32OutLen)
```

【描述】

使用 RSA 公钥对密文进行解密。

【参数】

参数名	描述	输入/输出
pstRsaDec	公钥解密结构	输入
pu8Input	待解密的输入数据	输入
u32InLen	待解密输入数据长度	输入
pu8Output	输出解密数据	输出
pu32OutLen	输出解密数据长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.15. ES_CIPHER_NSignVerify

【函数体】

```
ES_S32 ES_CIPHER_NsignVerify(  
    ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,  
    ES_U8 *pu8InData,  
    ES_U32 u32InDataLen,  
    ES_U8 *pu8InSign,  
    ES_U32 u32InSignLen,  
    ES_U8 *pu8Output,  
    ES_U32 *pu32OutLen)
```

【描述】

对 NSIGN bin 文件执行签名验证，验证通过后输出解密的签名结构信息。

【参数】

参数名	描述	输入/输出
pstNsignVerify	NSIGN 验证结构	输入
pu8InData	待签名验证的输入 payload	输入
u32InDataLen	输入数据长度	输入
pu8InSign	加密的签名结构信息	输入
u32InSignLen	加密的签名结构信息长度	输入
pu8Output	解密的签名结构信息（256 字节，固定长度）	输出
pu32OutLen	解密的签名结构信息长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.16. ES_CIPHER_ImageDec

【函数体】

```
ES_S32 ES_CIPHER_ImageDec (  
    ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,  
    ES_U8 *pu8InData,  
    ES_U32 u32InDataLen,  
    ES_U8 *pu8InSign,  
    ES_U32 u32InSignLen,  
    ES_U8 *pu8Output,  
    ES_U32 *pu32OutLen)
```

【描述】

对 NSIGN bin 文件执行签名验证并解密镜像，如果成功则输出解密的镜像数据。

【参数】

参数名	描述	输入/输出
pstNsignVerify	NSIGN 验证结构	输入
pu8InData	待签名验证的输入 payload	输入
u32InDataLen	payload 长度	输入
pu8InSign	加密的签名结构信息	输入
u32InSignLen	加密的签名结构信息长度	输入
pu8Output	解密的镜像数据	输出
pu32OutLen	解密的镜像数据长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

如果 pu8Output==NULL，则解密的镜像将被放置到 pu8InData 的地址。换句话说，输入 payload 将被解密后的 payload 覆盖。

2.17. ES_CIPHER_EcdhKeyGen

【函数体】

```
ES_S32 ES_CIPHER_EcdhKeyGen(  
    ES_CIPHER_ECC_CURVE_TYPE_E enEccCurve,  
    ES_CIPHER_ECDH_KEY_S *pstEcdhKey)
```

【描述】

生成一对 ECDH 公钥和私钥。

【参数】

参数名	描述	输入/输出
enEccCurve	椭圆曲线类型	输入
pstEcdhKey	输出的 ECDH 密钥，包含 ECC 公钥和私钥	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.18. ES_CIPHER_EcdhKeyAgree

【函数体】

```
ES_S32 ES_CIPHER_EcdhKeyAgree(  
    ES_CIPHER_ECC_CURVE_TYPE_E enEccCurve,  
    ES_CIPHER_ECDH_PUB_KEY_S *pstRemoteEcdhPubKey,  
    ES_CIPHER_ECDH_PRI_KEY_S *pstLocalEcdhPriKey,  
    ES_CIPHER_ECDH_PUB_KEY_S *pstOutSharedEcdhPubKey)
```

【描述】

ECDH 共享公钥协商，产生共享的 ECDH 公钥。

【参数】

参数名	描述	输入/输出
enEccCurve	椭圆曲线类型	输入
pstRemoteEcdhPubKey	远端 ECC 公钥结构	输入
pstLocalEcdhPriKey	本地 ECC 私钥结构	输入
pstOutSharedEcdhPubKey	协商的共享 ECC 公钥结构	输出

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

2.19. ES_CIPHER_EcdhKeyDerivation**【函数体】**

```
ES_S32 ES_CIPHER_EcdhKeyDerivation(
    ES_CIPHER_ECC_CURVE_TYPE_E enEccCurve,
    ES_CIPHER_ECDH_PUB_KEY_S *pstSharedEcdhPubKey,
    ES_U32 u32Rand,
    ES_U8 *pu8OutSymcKey)
```

【描述】

ECDH 密钥派生, 生成可作为对称算法密钥使用的密钥。

【参数】

参数名	描述	输入/输出
enEccCurve	椭圆曲线类型	输入
pstSharedEcdhPubKey	协商的共享 ECC 公钥结构	输入
u32Rand	本次协商中使用的随机数据	输入
pu8OutSymcKey	输出对称密钥	输出

【返回值】

返回值	描述
0	成功
非 0	失败, 参考 错误码

2.20. ES_CIPHER_PubKeyDownload**【函数体】**

```
ES_S32 ES_CIPHER_PubKeyDownload(  
    ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,  
    ES_U8 *pu8InData,  
    ES_U32 u32InDataLen,  
    ES_U8 *pu8InSign,  
    ES_U32 u32InSignLen)
```

【描述】

更新 CIPHER 硬件内部 RAM 中的公钥。

【参数】

参数名	描述	输入/输出
pstNsignVerify	NSIGN 验证结构	输入
pu8InData	待签名验证的输入 payload	输入
u32InDataLen	输入数据长度	输入
pu8InSign	加密的签名结构信息	输入
u32InSignLen	加密的签名结构信息长度	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

在此服务期间，pu8Indata 的内容将被解密数据覆盖。用户应妥善保存原始输入 payload 数据以防止数据丢失。正确的方法是 malloc 一个缓冲区，将原始 payload 数据复制到该缓冲区，并将该缓冲区作为输入 payload 使用。

2.21. ES_CIPHER_LoadableRegister

【函数体】

```
ES_S32 ES_CIPHER_LoadableRegister(  
    ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,  
    ES_U8 *pu8InData,  
    ES_U32 u32InDataLen,  
    ES_U8 *pu8InSign,  
    ES_U32 u32InSignLen,  
    ES_U32 *pu32ServiceId)
```

【描述】

可加载服务代码注册。

【参数】

参数名	描述	输入/输出
pstNsignVerify	NSIGN 验证结构	输入
pu8InData	待签名验证的输入 payload	输入
u32InDataLen	输入数据长度	输入
pu8InSign	加密的签名结构信息	输入
u32InSignLen	加密的签名结构信息长度	输入
pu32ServiceId	返回的服务 ID，将在 ES_CIPHER_LoadableIoctl 中使用	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.22. ES_CIPHER_LoadableUnRegister

【函数体】

```
ES_S32 ES_CIPHER_LoadableUnRegister(  
    ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,  
    ES_U32 u32ServiceId)
```

【描述】

可加载服务代码注销。

【参数】

参数名	描述	输入/输出
pstNsignVerify	NSIGN 验证结构	输入
u32ServiceId	服务 ID	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.23. ES_CIPHER_LoadableIoctl

【函数体】

```
ES_S32 ES_CIPHER_LoadableIoctl(  
    ES_CIPHER_NSIGN_VERIFY_S *pstNsignVerify,  
    ES_U32 u32ServiceId,
```



```
ES_U32 u32Cmd,  
ES_U32 u32ParamSize,  
ES_U8 *pu8Param,  
ES_U8 *pu8OutParam)
```

【描述】

可加载服务 ioctl 调用。

【参数】

参数名	描述	输入/输出
pstNsignVerify	NSIGN 验证结构	输入
u32ServiceId	之前注册的服务 ID	输入
u32Cmd	ioctl 命令	输入
u32ParamSize	参数大小	输入
pu8Param	参数	输入
pu8OutParam	带有签名数据的上下文	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

2.24. ES_CIPHER_RsaSign

【函数体】

```
ES_S32 ES_CIPHER_RsaSign(  
ES_CIPHER_RSA_PRI_CTRL_S *pstRsaSign,  
ES_U8 *pu8InData,  
ES_U32 u32InDataLen,  
ES_U8 *pu8OutSign,  
ES_U32 *pu32OutSignLen)
```

【描述】

RSA 签名函数。

【参数】

参数名	描述	输入/输出
pstRsaSign	RSA 私钥加密控制信息	输入
pu8InData	待签名的输入上下文	输入
u32InDataLen	待签名输入上下文长度	输入
pu8OutSign	输出签名信息	输出
pu32OutSignLen	签名信息长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

RSA 私钥只能是外部密钥，即由用户输入。

2.25. ES_CIPHER_RsaVerify

【函数体】

```
ES_S32 ES_CIPHER_RsaVerify(  
    ES_CIPHER_RSA_PUB_CTRL_S *pstRsaVerify,  
    ES_U8 *pu8InData,  
    ES_U32 u32InDataLen,  
    ES_U8 *pu8InSign,  
    ES_U32 u32InSignLen)
```

【描述】

RSA 验证函数。

【参数】

参数名	描述	输入/输出
pstRsaVerify	RSA 公钥解密控制信息	输入
pu8InData	待签名的输入上下文	输入
u32InDataLen	待签名输入上下文长度	输入
pu8InSign	签名信息	输入
u32InSignLen	签名信息长度	输入

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

提示

RSA 公钥可以是内部密钥（ROM、RAM）或外部密钥，即由用户输入。

2.26. ES_CIPHER_NsignFileParse

【函数体】

```
ES_S32 ES_CIPHER_NsignFileParse(
ES_U8 *pu8InFileData,
ES_U32 u32InFileDataLen,
ES_CIPHER_NSIGN_VERIFY_S *pstOutNsignVerify,
ES_U8 **ppu8OutSignAddr,
ES_U32 *pu32OutSignLen,
ES_U8 **ppu8OutPayloadAddr,
ES_U32 *pu32OutPayloadLen)
```

【描述】

从 NSIGN 输出 bin 文件中获取签名和 payload 的地址。

【参数】

参数名	描述	输入/输出
pu8InFileData	由 NSIGN 工具生成的文件数据地址	输入
u32InFileDataLen	文件数据长度	输入
pstOutNsignVerify	NSIGN 验证结构	输出
ppu8OutSignAddr	文件中的签名地址	输出
pu32OutSignLen	签名长度	输出
ppu8OutPayloadAddr	payload 地址	输出
pu32OutPayloadLen	payload 长度	输出

【返回值】

返回值	描述
0	成功
非 0	失败，参考 错误码

3. 数据结构

相关数据类型、数据结构定义如下：

- ES_HANDLE: 定义 CIPHER 的句柄类型。
- ES_CIPHER_ALG_MODE_E: 定义对称加密算法、摘要算法类型选择
- ES_CIPHER_ENC_KEY_LENGTH_E: 定义对称算法的 key 长度类型选择
- ES_CIPHER_SYMC_KEY_SRC_E: 定义对称算法的 key 的来源
- ES_CIPHER_CTRL_S: 定义对称加解密控制信息结构体
- ES_CIPHER_HASH_CTRL_S: 定义 HASH 算法控制信息结构体
- ES_CIPHER_RSA_ENC_SCHEME_E: 定义非对称算法的类型选择
- ES_CIPHER_RSA_KEY_SRC_E: 定义非对称算法的公钥来源以及外部公私钥选择
- ES_CIPHER_RSA_PUB_KEY_S: 定义 RSA 公钥结构体
- ES_CIPHER_RSA_PUB_CTRL_S: 定义 RSA 公钥加解密控制结构体
- ES_CIPHER_RSA_PRI_KEY_S: 定义 RSA 私钥结构体
- ES_CIPHER_RSA_PRI_CTRL_S: 定义 RSA 私钥加解密控制结构体
- ES_CIPHER_NSIGN_VERIFY_S: 定义验签过程中，使用的非对称算法和 Key 来源
- ES_CIPHER_ECC_CURVE_TYPE_E: 定义椭圆曲线类型选择
- ES_CIPHER_ECDH_KEY_S: 定义椭圆曲线公私钥对信息
- ES_CIPHER_ECDH_PUB_KEY_S: 定义椭圆曲线公钥信息
- ES_CIPHER_ECDH_PRI_KEY_S: 定义椭圆曲线私钥信息

3.1. ES_HANDLE

【说明】

定义 CIPHER 的句柄类型

【定义】

```
typedef unsigned int ES_HANDLE;
```

【成员】

无。

3.2. ES_CIPHER_ALG_MODE_E

【说明】

定义对称加密算法、摘要算法类型选择

【定义】

```
typedef enum esES_CIPHER_ALG_MODE_E
{
    ES_CIPHER_ALG_MODE_AES_ECB = 0x4,
    ES_CIPHER_ALG_MODE_AES_CBC,
    ES_CIPHER_ALG_MODE_AES_CTR,
    ES_CIPHER_ALG_MODE_AES_CCM,
    ES_CIPHER_ALG_MODE_AES_GCM,
    ES_CIPHER_ALG_MODE_AES_CFB,
    ES_CIPHER_ALG_MODE_AES_OFB,
    ES_CIPHER_ALG_MODE_AES_CSI,
    ES_CIPHER_ALG_MODE_AES_CS2,
    ES_CIPHER_ALG_MODE_AES_CS3,
    ES_CIPHER_ALG_MODE_3DES_CBC = 0x14,
    ES_CIPHER_ALG_MODE_3DES_ECB,
    ES_CIPHER_ALG_MODE_DES_CBC,
```

```

ES_CIPHER_ALG_MODE_DES_ECB,
ES_CIPHER_ALG_MODE_SM4_ECB = 0x1e,
ES_CIPHER_ALG_MODE_SM4_CBC,
ES_CIPHER_ALG_MODE_SM4_CFB,
ES_CIPHER_ALG_MODE_SM4_OFB,
ES_CIPHER_ALG_MODE_SM4_CTR,
ES_CIPHER_ALG_MODE_SM4_CCM,
ES_CIPHER_ALG_MODE_SM4_GCM,
ES_CIPHER_ALG_MODE_SM4_CS1,
ES_CIPHER_ALG_MODE_SM4_CS2,
ES_CIPHER_ALG_MODE_SM4_CS3,
ES_CIPHER_ALG_MODE_HASH_MD5 = 42,
ES_CIPHER_ALG_MODE_HMAC_MD5,
ES_CIPHER_ALG_MODE_HASH_SHA1,
ES_CIPHER_ALG_MODE_HMAC_SHA1,
ES_CIPHER_ALG_MODE_HASH_SHA224,
ES_CIPHER_ALG_MODE_HMAC_SHA224,
ES_CIPHER_ALG_MODE_HASH_SHA256,
ES_CIPHER_ALG_MODE_HMAC_SHA256,
ES_CIPHER_ALG_MODE_HASH_SHA384,
ES_CIPHER_ALG_MODE_HMAC_SHA384,
ES_CIPHER_ALG_MODE_HASH_SHA512,
ES_CIPHER_ALG_MODE_HMAC_SHA512,
ES_CIPHER_ALG_MODE_HASH_SHA512_224,
ES_CIPHER_ALG_MODE_HMAC_SHA512_224,
ES_CIPHER_ALG_MODE_HASH_SHA512_256,
ES_CIPHER_ALG_MODE_HMAC_SHA512_256,
ES_CIPHER_ALG_MODE_MAC_XCBC,
ES_CIPHER_ALG_MODE_MAC_CMAC,
ES_CIPHER_ALG_MODE_HASH_CRC32 = 66,
ES_CIPHER_ALG_MODE_HASH_SM3 = 80,
ES_CIPHER_ALG_MODE_HMAC_SM3,
ES_CIPHER_ALG_MODE_MAC_SM4_XCBC,
ES_CIPHER_ALG_MODE_MAC_SM4_CMAC,
} ES_CIPHER_ALG_MODE_E;

```

3.3. ES_CIPHER_ENC_KEY_LENGTH_E

【说明】

定义对称算法的 key 长度类型选择

【定义】

```

typedef enum esES_CIPHER_ENC_KEY_LENGTH_E
{
    ES_CIPHER_KEY_AES_128BIT = 16, /*< 128-bit key for the AES algorithm */
    ES_CIPHER_KEY_AES_192BIT = 24, /*< 192-bit key for the AES algorithm */
    ES_CIPHER_KEY_AES_256BIT = 32, /*< 256-bit key for the AES algorithm */
    ES_CIPHER_KEY_DES_3KEY = 24, /*< Three keys for the DES algorithm, TDES_KEY_192BIT */
    ES_CIPHER_KEY_SM_128BIT = 16, /*< SM4 */
    ES_CIPHER_KEY_DEFAULT = 8, /*< default key length, DES-8, SM4-16 */
    ES_CIPHER_KEY_LEN_MAX = 32,
    ES_CIPHER_KEY_INVALID = 0xffffffff,
} ES_CIPHER_ENC_KEY_LENGTH_E;

```

3.4. ES_CIPHER_SYMC_KEY_SRC_E

【说明】

定义对称算法的 key 的来源

【定义】

```
typedef enum esES_CIPHER_SYMC_KEY_SRC_E
{
    ES_CIPHER_SYMC_KEY_SRC_INTERNAL = 0x0, /**< Internal ROM Key*/
    ES_CIPHER_SYMC_KEY_SRC_USER = 0xFF, /**< External User Key*/
    ES_CIPHER_SYMC_KEY_SRC_INVALID = 0xffffffff,
} ES_CIPHER_SYMC_KEY_SRC_E;
```

3.5. ES_CIPHER_CTRL_S

【说明】

定义对称加解密控制信息结构体

【定义】

```
typedef struct esES_CIPHER_CTRL_S
{
    ES_U32 u32Key[8]; /**< Key input */
    ES_U32 u32IV[8]; /**< Initialization vector (IV), MAX IV_LEN is 32 Bytes */
    ES_CIPHER_SYMC_KEY_SRC_E enKeySrc; /**< Select key source: internal, or from u32Key[8] as external user key */
    ES_CIPHER_ALG_MODE_E enAlgMode; /**< Cipher algorithm */
    ES_CIPHER_ENC_KEY_LENGTH_E enKeyLen; /**< Key length */
    ES_U32 u32IvLen;
} ES_CIPHER_CTRL_S;
```

3.6. ES_CIPHER_HASH_CTRL_S

【说明】

定义 HASH 算法控制信息结构体

【定义】

```
typedef struct
{
    ES_U8 *pu8HMACKey;
    ES_U32 u32HMACKeyLen;
    ES_CIPHER_ALG_MODE_E enShaType;
} ES_CIPHER_HASH_CTRL_S;
```

3.7. ES_CIPHER_RSA_ENC_SCHEME_E

【说明】

定义非对称算法的类型选择

【定义】

```
typedef enum esES_CIPHER_RSA_ENC_SCHEME_E
{
    ES_CIPHER_RSA_SCHEME_RSA1024 = 0,
    ES_CIPHER_RSA_SCHEME_RSA2048,
    ES_CIPHER_RSA_SCHEME_RSA4096,
}
```

```
ES_CIPHER_RSA_SCHEME_ECDSA,
ES_CIPHER_RSA_SCHEME_PLAINTEXT,
}ES_CIPHER_RSA_ENC_SCHEME_E;
```

3.8. ES_CIPHER_RSA_KEY_SRC_E

【说明】

定义非对称算法的公钥来源以及外部公私钥选择

【定义】

```
typedef enum esES_CIPHER_RSA_KEY_SRC_E
{
    ES_CIPHER_RSA_PUB_KEY_SRC_OTP = 0x0, /**< Internal OTP Key*/
    ES_CIPHER_RSA_PUB_KEY_SRC_ROM_1, /**< Internal ROM Key 1*/
    ES_CIPHER_RSA_PUB_KEY_SRC_ROM_2, /**< Internal ROM Key 2*/
    ES_CIPHER_RSA_PUB_KEY_SRC_ROM_3, /**< Internal ROM Key 3*/
    ES_CIPHER_RSA_PUB_KEY_SRC_ROM_4, /**< Internal ROM Key 4*/
    ES_CIPHER_RSA_PUB_KEY_SRC_ROM_5, /**< Internal ROM Key 5*/
    ES_CIPHER_RSA_PUB_KEY_SRC_ROM_6, /**< Internal ROM Key 6*/
    ES_CIPHER_RSA_PUB_KEY_SRC_ROM_7, /**< Internal ROM Key 7*/
    ES_CIPHER_RSA_PUB_KEY_SRC_RAM = 8, /**< Internal ram Key*/
    ES_CIPHER_RSA_PRIV_KEY_SRC_EXT = 254, /**< External private Key*/
    ES_CIPHER_RSA_PUB_KEY_SRC_EXT = 255, /**< External public Key*/
    ES_CIPHER_RSA_KEY_SRC_INVALID = 0xffffffff,
}ES_CIPHER_RSA_KEY_SRC_E;
```

3.9. ES_CIPHER_RSA_PUB_KEY_S

【说明】

定义 RSA 公钥结构体

【定义】

```
typedef struct
{
    ES_U32 u32NLen; /**< length of public modulus(i.e, keylen), max value is 512Byte*/
    ES_U32 u32E; /**< public exponent */
    ES_U8 *pu8N; /**< point to public modulus */
}ES_CIPHER_RSA_PUB_KEY_S;
```

3.10. ES_CIPHER_RSA_PUB_CTRL_S

【说明】

定义 RSA 公钥加解密控制结构体

【定义】

```
typedef struct
{
    ES_CIPHER_RSA_ENC_SCHEME_E enScheme; /** RSA encryption scheme*/
    ES_CIPHER_RSA_KEY_SRC_E enKeySrc; /** RSA key source*/
    ES_CIPHER_RSA_PUB_KEY_S stPubKey; /** RSA public key struct */
}ES_CIPHER_RSA_PUB_CTRL_S;
```

3.11. ES_CIPHER_RSA_PRI_KEY_S

【说明】

定义 RSA 私钥结构体

【定义】

```
typedef struct
{
    ES_U32 u32NLen; /**< length of public modulus(i.e, keylen), max value is 512Byte*/
    ES_U8 *pu8D; /**< private exponent */
    ES_U8 *pu8N; /**< public modulus */
}ES_CIPHER_RSA_PRI_KEY_S;
```

3.12. ES_CIPHER_RSA_PRI_CTRL_S

【说明】

定义 RSA 私钥加解密控制结构体

【定义】

```
typedef struct
{
    ES_CIPHER_RSA_ENC_SCHEME_E enScheme; /**< RSA encryption scheme */
    ES_CIPHER_RSA_KEY_SRC_E enKeySrc; /**< RSA key source*/
    ES_CIPHER_RSA_PRI_KEY_S stPriKey; /**< RSA private key struct */
}ES_CIPHER_RSA_PRI_CTRL_S;
```

3.13. ES_CIPHER_NSIGN_VERIFY_S

【说明】

定义验签过程中，使用的非对称算法和 Key 来源结构体

【定义】

```
typedef struct
{
    ES_CIPHER_RSA_ENC_SCHEME_E enScheme; /**< RSA signature scheme*/
    ES_CIPHER_RSA_KEY_SRC_E enKeySrc; /**< RSA key source*/
}ES_CIPHER_NSIGN_VERIFY_S;
```

3.14. ES_CIPHER_ECC_CURVE_TYPE_E

【说明】

定义椭圆曲线类型选择

【定义】

```
typedef enum esES_CIPHER_ECC_CURVE_TYPE_E
{
    ES_CIPHER_PKA_SW_CURVE_NIST_P256 = 0,
    ES_CIPHER_PKA_SW_CURVE_NIST_P384,
    ES_CIPHER_PKA_SW_CURVE_NIST_P521,
    ES_CIPHER_PKA_SW_CURVE_BRAINPOOL_P256R1,
    ES_CIPHER_PKA_SW_CURVE_BRAINPOOL_P384R1,
```



```
ES_CIPHER_PKA_SW_CURVE_BRAINPOOL_P512R1,  
ES_CIPHER_PKA_SW_CURVE_INVALID  
}ES_CIPHER_ECC_CURVE_TYPE_E;
```

3.15. ES_CIPHER_ECDH_KEY_S

【说明】

定义椭圆曲线公私钥对信息

【定义】

```
typedef struct  
{  
    ES_U32 keylen;  
    ES_U8 *pu8X;  
    ES_U8 *pu8Y;  
    ES_U8 *pu8Z;  
}ES_CIPHER_ECDH_KEY_S;
```

3.16. ES_CIPHER_ECDH_PUB_KEY_S

【说明】

定义椭圆曲线公钥信息

【定义】

```
typedef struct  
{  
    ES_U32 keylen;  
    ES_U8 *pu8X;  
    ES_U8 *pu8Y;  
}ES_CIPHER_ECDH_PUB_KEY_S;
```

3.17. ES_CIPHER_ECDH_PRI_KEY_S

【说明】

定义椭圆曲线私钥信息

【定义】

```
typedef struct  
{  
    ES_U32 keylen;  
    ES_U8 *pu8Z;  
}ES_CIPHER_ECDH_PRI_KEY_S;
```

4. 错误码

Cipher 相关 API 错误码如下表所示。

错误码	宏定义	描述
0xA0136003	ES_ERR_CIPHER_INVALID_PARA	无效参数
0xA0136040	ES_ERR_CIPHER_NOT_INIT	CIPHER 未初始化
0xA0136041	ES_ERR_CIPHER_FAILED_INIT	CIPHER 初始化失败
0xA0136042	ES_ERR_CIPHER_OVERFLOW	CIPHER 初始化次数超过限制
0xA0136043	ES_ERR_CIPHER_INVALID_HANDLE	对称加密句柄 ID 无效
0xA0136044	ES_ERR_CIPHER_FAILED_GETHANDLE	获取句柄失败
0xA0136045	ES_ERR_CIPHER_FAILED_RELEASE_HANDLE	释放句柄失败
0xA0136046	ES_ERR_CIPHER_HW_ERROR	CIPHER 硬件错误，发送消息到 CIPHER 硬件失败
0xA0136047	ES_ERR_CIPHER_SERVICE_FAILED	服务发生错误
0xA0136048	ES_ERR_CIPHER_IOVA	IOVA 分配或释放失败